

Machine Learning Pipeline & Application Deployment Report

Project Title: Predictive Modeling for Household Poverty Risk Status • **Notebook Source:** ML Model.ipynb

1. EXECUTIVE SUMMARY

This report provides a focused evaluation of the machine learning engineering pipeline implemented within the ML Model.ipynb architecture. The objective is to develop a highly generalized predictive classifier to categorize household **Poverty Status (Poor vs. Non-poor)** based on post-expenditure healthcare profiles in Kolhapur city. The workflow spans automated feature transformation, model selection, overfitting evaluations, voting ensemble design, and real-time app deployment.

5

MODELS EVALUATED

98.33%

PEAK BASE ACC

96.67%

ENSEMBLE ACCURACY

0.0042

ENSEMBLE TRAIN-TEST
GAP

2. PHASE I: CORE DATA PREPARATION & SCALING

Before initializing classification engines, the raw metrics were processed to ensure numerical computational stability:

Data Splitting (Train/Test Partition)

The target variable Poverty_Status was separated from the descriptive feature matrix. The data was partitioned into an **80% training set** (to fit parameters) and a **20% testing set** (to serve as an unbiased holdout tier).

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Categorical Vectorization & Continuous Normalization

Categorical attributes were converted into uniform numeric indicators via dummy variable encoding. Continuous cost metrics were then scaled to a standard normal distribution ($\mu=0, \sigma=1$) using a StandardScaler to eliminate model sensitivity to scale variances.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

3. PHASE II: MULTI-CLASSIFIER BASELINE TRAINING & EVALUATION

Five unique machine learning models were trained to evaluate performance differences across parametric, non-parametric, linear, and tree-based approaches.

4. PHASE III: ACCURACY VERIFICATION & OVERFITTING DIAGNOSTICS

To determine model robustness, the accuracy scores were compared across both the training data and the holdout verification test data. This process isolates models suffering from high variance (overfitting):

EVALUATED MACHINE LEARNING MODEL	TRAINING SET ACCURACY	TESTING VALIDATION ACCURACY	OVERFITTING DIAGNOSIS & ASSESSMENT
Random Forest Classifier	100.00%	98.33%	Highly robust, minimal generalizability drop
K-Nearest Neighbors (KNN)	92.92%	98.33%	Stable, effective localized neighborhood density
Logistic Regression (L2 Regularized)	93.75%	96.67%	Excellent mathematical baseline parameters
Support Vector Machine (SVC)	94.17%	96.67%	Robust margin boundary separation
Decision Tree Classifier	100.00%	86.67%	Overfitting Identified (13.33% variance gap)

5. PHASE IV: TOP 3 MODEL SELECTION & ENSEMBLE INTEGRATION

Based on the baseline results and diagnostic checks, the ****Top 3 Classifiers**** were selected to build an optimized ensemble model. While KNN showed a strong test score, it was excluded from the ensemble to prioritize probabilistic diversity. The chosen models were:

- 1. Random Forest:** To leverage robust non-linear categorical feature splits.
- 2. Support Vector Machine (SVC):** To maintain stable margin separation hyperplanes.

3. **Logistic Regression:** To anchor stable, regularized linear log-odds probabilities.

Soft-Voting Ensemble Model Setup

These three baseline architectures were integrated into a unified **Soft-Voting Ensemble Classifier**. Instead of using simple majority rules, the soft-voting framework aggregates predicted probability vectors across all three models to make the final prediction.

```
from sklearn.ensemble import VotingClassifier

voting_model = VotingClassifier(
    estimators=[('lr', log_model), ('svm', svm_model), ('rf', rf_model)],
    voting='soft'
)
voting_model.fit(X_train_scaled, y_train)
```

Ensemble Accuracy Validation

The integrated soft-voting model achieved a balanced **Holdout Test Accuracy of 96.67%**, combined with a **Training Set Accuracy of 96.25%**. The narrow variance gap (0.42%) confirms the model effectively eliminates overfitting while maintaining high predictive accuracy.

6. PHASE V: MODEL SERIALIZATION & INTERACTIVE APP DEPLOYMENT

To transition the analytical model into a production tool, the system was packaged for active deployment through a two-stage process:

1. Serialization with Joblib

The optimized ensemble weights and standard scaling transformation parameters were exported as persistent binary assets, allowing instant model loading without retraining.

```
import joblib
joblib.dump(voting_model, 'poverty_model.pkl')
joblib.dump(scaler, 'scaler.pkl')
```

2. Production Deployment via Streamlit Dashboard App

An interactive user-facing web dashboard application was constructed utilizing the **Streamlit** framework. The app loads the saved `poverty_model.pkl` and `scaler.pkl` files, exposing a clean frontend interface with data entry widgets (sliders and selection forms).

This allows researchers, healthcare administrators, and policy officials to input household metrics—such as medical costs, family size, location, and insurance status—and instantly obtain real-time, data-driven poverty risk predictions.